

# **LAPORAN PRAKTIKUM**

## **JOBSHEET 5 - JAVASCRIPT DOM DAN EVENT**



**Disusun Oleh:**

Nama : Mohammad Hafidz Rafi' Rabbani  
NIM : 254107020084  
Kelas : 2H  
No. Absen : 14  
Repository : [github.com/hafidzrafi/DnPWeb2026](https://github.com/hafidzrafi/DnPWeb2026)

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**JURUSAN TEKNOLOGI INFORMASI**  
**POLITEKNIK NEGERI MALANG**  
**2026**

# Daftar Isi

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>1. Tujuan .....</b>                                                    | <b>3</b>  |
| <b>2. Praktikum .....</b>                                                 | <b>3</b>  |
| 2.1. Percobaan 1: Menu Hamburger dengan JavaScript .....                  | 3         |
| 2.1.1. 1.1 Analisis Mekanisme classList.toggle .....                      | 3         |
| 2.1.2. 1.2 Keunggulan Arsitektur Dibanding Checkbox Hack .....            | 4         |
| 2.1.3. Langkah-langkah .....                                              | 5         |
| 2.1.4. Pertanyaan .....                                                   | 7         |
| 2.2. Percobaan 2: Konfirmasi Hapus Data Front-End .....                   | 8         |
| 2.2.1. 2.1 Mekanisme DOM Traversal closest() dan remove() .....           | 8         |
| 2.2.2. Langkah-langkah .....                                              | 9         |
| 2.2.3. Pertanyaan .....                                                   | 10        |
| 2.3. Percobaan 3: Filter dan Pencarian Tabel Real-Time .....              | 11        |
| 2.3.1. 3.1 Implementasi Event Listener keyup .....                        | 11        |
| 2.3.2. 3.2 Analisis String Matching & Display Toggle .....                | 11        |
| 2.3.3. Langkah-langkah .....                                              | 12        |
| 2.3.4. Pertanyaan .....                                                   | 13        |
| 2.4. Percobaan 4: Validasi Form Sisi Klien (Client-Side Validation) ..... | 14        |
| 2.4.1. 4.1 Konstruksi Pesan Error Dinamis .....                           | 14        |
| 2.4.2. 4.2 Pola Clean-State & Pembatalan Submit (e.preventDefault) .....  | 14        |
| 2.4.3. Langkah-langkah .....                                              | 16        |
| 2.4.4. Pertanyaan .....                                                   | 17        |
| <b>3. Tugas .....</b>                                                     | <b>18</b> |
| 3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 5 .....                | 18        |

## 1. Tujuan

- Memahami konsep dasar Document Object Model (DOM) sebagai pohon objek (tree hierarchy) di memori browser yang merepresentasikan dokumen HTML.
- Menguasai teknik seleksi elemen DOM menggunakan metode modern seperti `document.getElementById()` dan `document.querySelectorAll()`.
- Menerapkan pemrograman berbasis kejadian (event-driven programming) melalui penambahan Event Listener pada elemen antarmuka.
- Menggantikan teknik CSS Checkbox Hack dengan manipulasi status antarmuka berbasis `classList.toggle()` pada menu navigasi responsif (hamburger menu).
- Mengimplementasikan konfirmasi interaktif penghapusan baris data tabel secara lokal di sisi klien menggunakan DOM traversal `closest()` dan method `remove()`.
- Membangun fitur pencarian dan penyaringan data tabel secara real-time berdasarkan input keyboard pengguna (event `keyup/input`) dan manipulasi properti `style.display`.
- Merancang sistem validasi form di sisi klien (client-side validation) dengan pencegahan aksi bawaan form via `e.preventDefault()` serta penyisipan pesan error dinamis menggunakan `insertAdjacentElement()`.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: <https://github.com/hafidzrafi/DnPWeb2026>.

## 2. Praktikum

### 2.1. Percobaan 1: Menu Hamburger dengan JavaScript

Pada percobaan pertama, mekanisme navigasi mobile (**hamburger menu**) yang sebelumnya diimplementasikan pada Jobsheet 3 menggunakan teknik **Checkbox Hack** CSS murni (`#nav-toggle:checked ~ nav`) kini dimigrasikan menggunakan tombol semantik `<button id="nav-toggle-btn">` yang dikendalikan oleh fungsi JavaScript `initNavToggle()`.

#### 2.1.1. 1.1 Analisis Mekanisme `classList.toggle`

Fungsi `initNavToggle()` bekerja dengan mencari elemen tombol pemicu dan elemen `<nav>` di dalam `<header>`. Saat pengguna menekan tombol, method `nav.classList.toggle("nav-open")` memeriksa apakah class `.nav-open` sudah terpasang pada elemen navigasi. Jika belum ada maka ditambahkan, dan jika sudah ada maka dicabut:

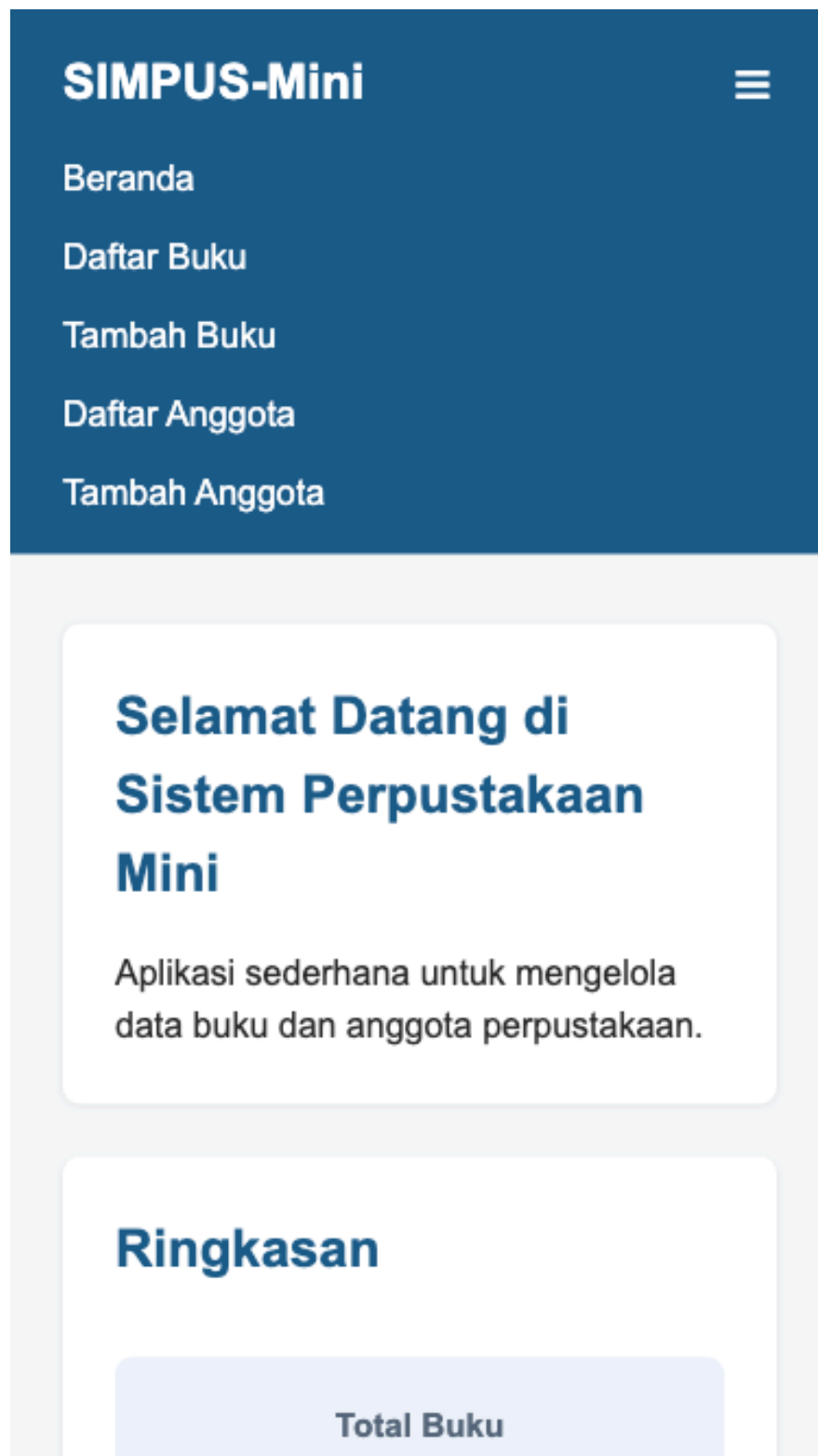
```
function initNavToggle() {
  const toggleBtn = document.getElementById("nav-toggle-btn");
  const nav = document.querySelector("header nav");
  if (!toggleBtn || !nav) return;

  toggleBtn.addEventListener("click", function() {
    nav.classList.toggle("nav-open");
  });
}
```

### 2.1.2. 1.2 Keunggulan Arsitektur Dibanding Checkbox Hack

Migrasi ke JavaScript memberikan dua keunggulan rekayasa yang mendasar:

1. **Aksesibilitas dan Semantik Web (A11y):** Menggunakan elemen `<button>` resmi yang secara bawaan dapat menerima fokus keyboard (tombol Tab), dapat diaktifkan menggunakan tombol Spasi/Enter, dan terbaca secara akurat oleh pembaca layar (**screen reader**).
2. **Pemisahan Peran (Separation of Concerns):** Menghilangkan ketergantungan rapuh pada selector CSS saudara (`~`) dan pseudo-class `:checked`. CSS kini murni bertugas mengatur tata letak visual elemen yang memiliki class `.nav-open` (`display: block`).



**Gambar 1.1:** Antarmuka Navigasi Mobile Terbuka saat Tombol Hamburger Diklik  
( `nav.classList.toggle("nav-open")` ).

### 2.1.3. Langkah-langkah

1. Membuat berkas JavaScript terpusat pada direktori `assets/js/app.js`.

2. Menggantikan elemen `<input type="checkbox" id="nav-toggle">` pada header HTML dengan `<button id="nav-toggle-btn" class="nav-toggle-label">≡</button>`.
3. Menghubungkan berkas script menggunakan tag `<script src="../assets/js/app.js"></script>` tepat sebelum penutup tag `</body>`.
4. Menguji klik tombol hamburger pada viewport mobile (lebar 375px) dan memverifikasi kemunculan menu vertikal.

#### 2.1.4. Pertanyaan

1. Mengapa penempatan tag `<script>` direkomendasikan diletakkan tepat sebelum penutup `</body>` ?
  - Penempatan tag `<script>` di akhir `</body>` mencegah terjadinya **parser blocking**. Ketika browser mem-parse dokumen HTML dari atas ke bawah, penempatan script di dalam `<head>` akan menghentikan pembentukan DOM untuk mengunduh dan mengeksekusi JavaScript. Jika script mengakses elemen halaman saat DOM belum selesai dibangun, fungsi seperti `document.getElementById()` akan menghasilkan nilai `null` dan memicu error **TypeError**. Meletakkan script sebelum `</body>` memastikan seluruh elemen antarmuka selesai dirender terlebih dahulu sehingga halaman terasa cepat dimuat (**First Contentful Paint** optimal).

## 2.2. Percobaan 2: Konfirmasi Hapus Data Front-End

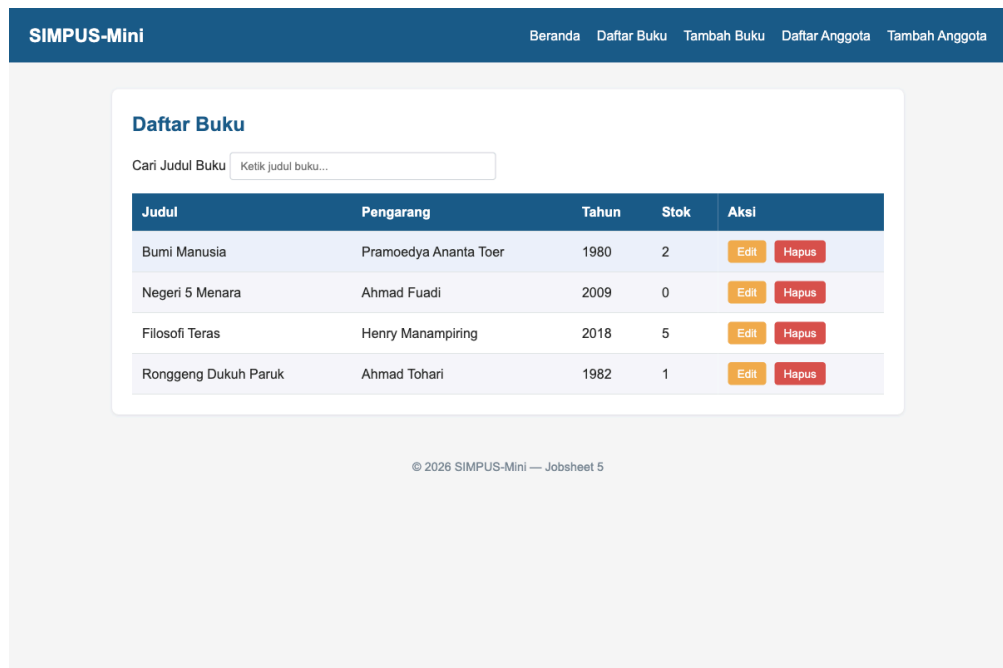
Percobaan kedua menerapkan interaktivitas konfirmasi penghapusan data pada tabel buku dan anggota. Karena backend database baru akan diperkenalkan pada Jobsheet 8, aksi penghapusan pada Jobsheet 5 ini beroperasi di level **front-end DOM manipulation**.

### 2.2.1. 2.1 Mekanisme DOM Traversal `closest()` dan `remove()`

Saat pengguna menekan tombol dengan class `.btn-hapus`, event listener menangkap target klik dan menelusuri hierarki pohon DOM ke atas (**ancestor traversal**) menggunakan method `btn.closest("tr")`. Method ini secara cerdas mencari elemen baris tabel pembungkus terdekat tanpa peduli berapa lapis tag yang membungkus tombol tersebut:

```
function initHapusConfirm() {
  document.querySelectorAll(".btn-hapus").forEach(function(btn) {
    btn.addEventListener("click", function() {
      const row = btn.closest("tr");
      const nama = row ? row.querySelector("td").textContent : "data ini";
      const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
      if (yakin && row) {
        row.remove();
      }
    });
  });
}
```

Method bawaan browser `confirm()` menampilkan dialog modal konfirmasi sinkron yang memblokir eksekusi sementara hingga pengguna memilih OK (`true`) atau Batal (`false`). Jika disetujui, `row.remove()` mencabut elemen `<tr>` secara langsung dari Document Object Model, memicu proses **reflow** dan **repaint** pada browser.



**Gambar 1.2:** Baris Pertama Tabel Berhasil Dihapus dari DOM setelah Konfirmasi Dialog `confirm()` Disetujui.

### 2.2.2. Langkah-langkah

1. Menambahkan class `.btn-hapus` pada setiap tombol aksi Hapus di tabel daftar buku dan anggota.
2. Mengimplementasikan fungsi `initHapusConfirm()` dengan perulangan `querySelectorAll().forEach()`.
3. Menguji penekanan tombol Hapus pada salah satu baris buku dan memverifikasi dialog konfirmasi serta hilangnya baris dari layar.

### 2.2.3. Pertanyaan

1. Apa perbedaan mendasar antara menghapus elemen menggunakan `row.remove()` dibandingkan menyembunyikannya menggunakan `row.style.display = "none"`?
  - `row.remove()` secara fisik mencabut node elemen beserta seluruh turunan dan event listener-nya dari memori Document Object Model (DOM tree). Elemen tersebut benar-benar hilang dari struktur dokumen. Sebaliknya, `row.style.display = "none"` hanya mengubah properti tata letak visual (**rendering tree**) sehingga elemen tidak digambar di layar, namun node tersebut tetap ada di dalam memori DOM dan tetap bisa ditemukan melalui query selector JavaScript.

## 2.3. Percobaan 3: Filter dan Pencarian Tabel Real-Time

Percobaan ketiga membangun fitur pencarian instan pada tabel tanpa memicu pemuatan ulang halaman (**zero page-reload**). Pengguna cukup mengetik kata kunci pada kolom input, dan baris tabel yang tidak sesuai akan disaring secara otomatis seiring tombol keyboard ditekan.

### 2.3.1. 3.1 Implementasi Event Listener keyup

Fungsi `initTableFilter()` mendengarkan kejadian keyboard pada `#search-input`:

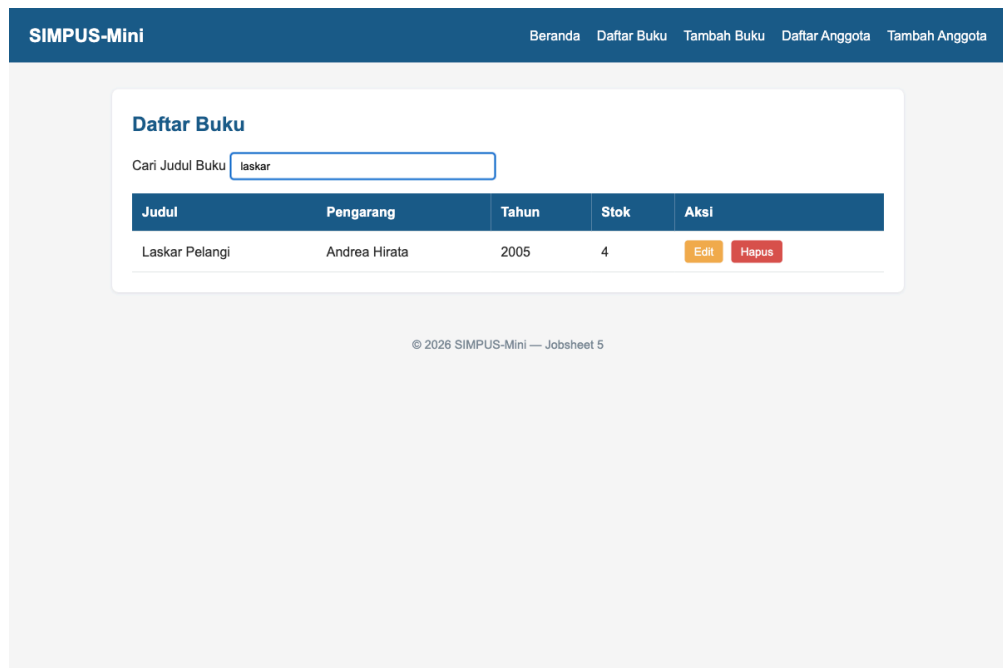
```
function initTableFilter() {
  const input = document.getElementById("search-input");
  const table = document.querySelector(".table-responsive table");
  if (!input || !table) return;

  input.addEventListener("keyup", function() {
    const keyword = input.value.toLowerCase();
    const rows = table.querySelectorAll("tbody tr");
    rows.forEach(function(row) {
      const teks = row.textContent.toLowerCase();
      row.style.display = teks.includes(keyword) ? "" : "none";
    });
  });
}
```

### 2.3.2. 3.2 Analisis String Matching & Display Toggle

Properti `row.textContent` secara otomatis menggabungkan seluruh teks dari kolom Judul, Pengarang, Tahun, hingga Kategori pada baris tersebut. Normalisasi string menggunakan `.toLowerCase()` memastikan pencarian bersifat **case-insensitive**.

Saat kata kunci cocok (`teks.includes(keyword)` bernilai `true`), baris diatur ke `display = ""` agar kembali ke aturan stylesheet default (`display: table-row`), bukan dipaksa menjadi `display: block` yang berpotensi merusak tata letak kisi tabel HTML.



**Gambar 1.3:** Hasil Penyaringan Real-Time Kata Kunci “laskar” Menampilkan Buku Terkait dan Menyembunyikan Baris Lainnya.

### 2.3.3. Langkah-langkah

1. Menambahkan elemen input pencarian `<input type="search" id="search-input">` di atas tabel data.
2. Mengimplementasikan fungsi `initTableFilter()` yang mendengarkan event keyboard.
3. Mengetikkan berbagai kata kunci (sebagian kata, huruf besar/kecil) dan mengamati perilaku penyaringan baris tabel secara real-time.

#### 2.3.4. Pertanyaan

1. Mengapa pada kasus pencarian teks modern, event `input` lebih direkomendasikan daripada event `keyup`?
  - Event `keyup` hanya terpicu saat tombol fisik pada keyboard dilepas. Jika pengguna memasukkan teks dengan cara melakukan klik kanan lalu memilih **Paste** menggunakan mouse, memilih kata dari saran **autocomplete**, atau menekan tombol silang pembersih teks pada input pencarian, event `keyup` tidak akan pernah terpicu sehingga tabel gagal tersaring. Sebaliknya, event `input` dirancang untuk menangkap setiap perubahan nilai masukan (**value mutation**) tanpa peduli dari mana sumber aksi tersebut berasal.

## 2.4. Percobaan 4: Validasi Form Sisi Klien (Client-Side Validation)

Percobaan keempat mengamankan integritas input pengguna sebelum data dikirimkan. Form Tambah Buku dan Tambah Anggota diproteksi dengan aturan validasi field wajib, rentang tahun terbit (1900–2026), serta kuantitas stok non-negatif.

### 2.4.1. 4.1 Konstruksi Pesan Error Dinamis

Pesan error dibuat secara dinamis menggunakan `document.createElement("span")` dengan class CSS `.error` dan disisipkan tepat di bawah field yang bermasalah menggunakan method `insertAdjacentElement("afterend", span)`:

```
function tampilkanError(input, pesan) {
    hapusError(input);
    const span = document.createElement("span");
    span.className = "error";
    span.textContent = pesan;
    input.insertAdjacentElement("afterend", span);
}

function hapusError(input) {
    const next = input.nextElementSibling;
    if (next && next.classList.contains("error")) {
        next.remove();
    }
}
```

### 2.4.2. 4.2 Pola Clean-State & Pembatalan Submit (e.preventDefault)

Fungsi `tampilkanError()` selalu mengeksekusi `hapusError(input)` terlebih dahulu untuk mencegah duplikasi pesan error bertumpuk saat tombol submit ditekan berkali-kali.

Apabila terdapat minimal satu field yang tidak valid (`valid === false`), method `e.preventDefault()` wajib dipanggil untuk membatalkan pengiriman HTTP dan refresh halaman bawaan browser:

The screenshot shows the 'Tambah Buku' (Add Book) form in the SIMPUS-Mini application. The form is contained within a white box with a light gray border. At the top of the box is the title 'Tambah Buku' in bold blue text. Below the title are several input fields, each with a label above it: 'Judul', 'Pengarang', 'Tahun Terbit', 'ISBN', 'Stok', and 'Kategori'. The 'Judul' field has a red error message 'Field ini wajib diisi.' below it. The 'Pengarang' field also has a red error message 'Field ini wajib diisi.' below it. The 'Tahun Terbit' field has a red error message 'Tahun harus di antara 1900-2026.' below it. The 'Stok' field has a red error message 'Stok tidak boleh negatif.' below it. The 'Kategori' field is a dropdown menu with 'Fiksi' selected. At the bottom of the form is a blue 'Simpan' button. The background of the page is a light gray, and the top navigation bar is dark blue with white text links: 'Beranda', 'Daftar Buku', 'Tambah Buku', 'Daftar Anggota', and 'Tambah Anggota'.

**Gambar 1.4:** Tampilan Pesan Error Validasi Inline pada Form Tambah Buku saat Terjadi Pelanggaran Validasi Input.

The screenshot shows the 'Tambah Anggota' (Add Member) form in the SIMPUS-Mini application. The form is contained within a white box with a light gray border. At the top of the box is the title 'Tambah Anggota' in bold blue text. Below the title are four input fields, each with a label above it: 'Nama', 'No. Anggota', 'Alamat', and 'No. HP'. The 'Nama' field has a red error message 'Field ini wajib diisi.' below it. The 'No. Anggota' field is empty. The 'Alamat' field is empty. The 'No. HP' field is empty. At the bottom of the form is a blue 'Simpan' button. The background of the page is a light gray, and the top navigation bar is dark blue with white text links: 'Beranda', 'Daftar Buku', 'Tambah Buku', 'Daftar Anggota', and 'Tambah Anggota'. At the bottom of the page, there is a small copyright notice: '© 2026 SIMPUS-Mini — Jobsheet 5'.

**Gambar 1.5:** Tampilan Pesan Error Validasi Inline pada Form Tambah Anggota saat Field Nama Dikosongkan.

### 2.4.3. Langkah-langkah

1. Menambahkan atribut `id="form-tambah"` pada tag `<form>` di `books/tambah.html` dan `members/tambah.html`.
2. Menambahkan aturan gaya CSS untuk class `.error` dengan warna teks merah semantik `#d9534f`.
3. Mengimplementasikan fungsi pembantu `tampilkanError()`, `hapusError()`, dan fungsi pengendali utama `initValidasiForm()`.
4. Menguji pengiriman form dengan field kosong, angka tahun di luar rentang, dan stok negatif untuk memverifikasi penolakan form serta kemunculan pesan error inline.

#### 2.4.4. Pertanyaan

1. Apa yang terjadi apabila `e.preventDefault()` tidak dipanggil saat form mendeteksi data yang tidak valid?
  - Jika `e.preventDefault()` tidak dipanggil, browser akan tetap menjalankan aksi bawaan (**default submit action**) yaitu merefresh/memuat ulang halaman. Akibatnya, elemen pesan error yang baru saja disisipkan oleh JavaScript akan langsung musnah seketika dan seluruh isian form akan tereset ke kondisi semula, sehingga pengguna tidak dapat membaca alasan mengapa data yang mereka masukkan ditolak.

### 3. Tugas

#### 3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 5

Berikut adalah evaluasi kritis dan pembahasan mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 5:

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <p><b>Soal:</b> Mengapa penempatan tag <code>&lt;script src="..."&gt;</code> direkomendasikan diletakkan tepat sebelum tag penutup <code>&lt;/body&gt;</code> dan bukan di dalam <code>&lt;head&gt;</code>?</p> <p><b>Jawaban:</b><br/>Proses parsing dokumen HTML oleh mesin peramban (<b>browser engine</b>) berlangsung secara sekuensial dari baris teratas ke baris terbawah. Ketika tag <code>&lt;script&gt;</code> diletakkan di dalam <code>&lt;head&gt;</code>, browser mengalami <b>parser blocking</b> di mana pembentukan pohon DOM dihentikan sementara hingga berkas JavaScript selesai diunduh dan dieksekusi. Karena elemen-elemen di dalam tag <code>&lt;body&gt;</code> belum terbaca, pemanggilan selektor seperti <code>document.getElementById("form-tambah")</code> akan mengembalikan nilai <code>null</code>. Menempatkan tag script tepat sebelum penutup <code>&lt;/body&gt;</code> menjamin seluruh Document Object Model telah terkonstruksi secara utuh, mempercepat metrik <b>First Contentful Paint (FCP)</b>, serta mencegah timbulnya <b>runtime error</b>. Alternatif modern pada web engineering adalah meletakkannya di <code>&lt;head&gt;</code> dengan menambahkan atribut <code>defer</code> agar pengunduhan berjalan asinkron dan dieksekusi tepat sebelum event <code>DOMContentLoaded</code>.</p> |
| 2  | <p><b>Soal:</b> Bagaimana cara kerja fungsi <code>initNavToggle()</code> menggunakan <code>classList.toggle("nav-open")</code> dan apa kelebihanannya dibandingkan teknik Checkbox Hack CSS murni pada Jobsheet 3?</p> <p><b>Jawaban:</b><br/>Fungsi <code>initNavToggle()</code> memasang penanganan event <code>click</code> pada elemen tombol <code>#nav-toggle-btn</code>. Setiap kali tombol ditekan, method <code>classList.toggle("nav-open")</code> mengevaluasi keberadaan</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>class <code>.nav-open</code> pada elemen <code>&lt;nav&gt;</code>. Jika class telah ada maka akan dicabut, dan jika belum ada maka akan ditambahkan, sehingga memicu aturan CSS <code>header nav.nav-open { display: block; }</code>.</p> <p>Kelebihan utama pendekatan ini meliputi:</p> <ol style="list-style-type: none"> <li>1. <b>Aksesibilitas Semantik (A11y):</b> Menggunakan tag semantik <code>&lt;button&gt;</code> yang dapat diakses penuh melalui keyboard (navigasi Tab, tombol Enter/Spasi) dan dikenali oleh teknologi asistif (<b>screen reader</b>).</li> <li>2. <b>Pemisahan Perhatian (Separation of Concerns):</b> Memisahkan state interaksi (dikelola oleh JavaScript) dari aturan presentasi visual (dikelola oleh CSS), serta menyederhanakan kode dengan mengeliminasi selector kombinator saudara (<code>~</code>) yang rumit pada Checkbox Hack.</li> </ol>                                                                                                                                                                                                                                                                                                        |
| 3  | <p><b>Soal:</b> Bagaimana mekanisme event <code>keyup</code> pada <code>initTableFilter()</code> menyaring baris data tabel secara real-time? Jelaskan penggunaan properti <code>textContent</code> dan manipulasi CSS <code>style.display</code>.</p> <p><b>Jawaban:</b></p> <p>Setiap kali pengguna melepaskan tombol keyboard di dalam <code>#search-input</code>, event <code>keyup</code> terpicu. Fungsi mengambil nilai input dan menormalisasinya menjadi huruf kecil melalui method <code>.toLowerCase()</code>. Selanjutnya, fungsi melakukan perulangan pada seluruh baris <code>&lt;tr&gt;</code> di dalam <code>&lt;tbody&gt;</code>. Properti <code>row.textContent</code> mengekstrak seluruh gabungan teks dari semua sel <code>&lt;td&gt;</code> pada baris tersebut. String teks ini juga dinormalisasi ke huruf kecil lalu diuji kecocokannya menggunakan method <code>.includes(keyword)</code>. Apabila cocok, tampilan baris dikembalikan dengan menyetel <code>row.style.display = ""</code> (mengikuti aturan stylesheet default yaitu <code>display: table-row</code>). Apabila tidak cocok, baris disembunyikan menggunakan <code>row.style.display = "none"</code>.</p> |
| 4  | <p><b>Soal:</b> Mengapa method <code>e.preventDefault()</code> wajib dipanggil saat proses validasi form (<code>initValidasiForm</code>) mendeteksi adanya data</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>input yang tidak valid? Apa yang terjadi jika method tersebut tidak dipanggil?</p> <p><b>Jawaban:</b></p> <p>Perilaku standar (<b>default behavior</b>) dari event <code>submit</code> pada elemen <code>&lt;form&gt;</code> adalah mengirimkan data ke URL target dan memicu pemuatan ulang (<b>reload</b>) halaman. Jika <code>e.preventDefault()</code> tidak dipanggil ketika input terbukti tidak valid, browser akan tetap mengeksekusi reload halaman. Akibatnya, seluruh elemen pesan kesalahan <code>&lt;span class="error"&gt;</code> yang baru saja disisipkan ke dalam pohon DOM akan langsung musnah seketika dan form kembali kosong. Pemanggilan <code>e.preventDefault()</code> membatalkan aksi reload tersebut sehingga pengguna tetap berada pada halaman aktif dan dapat membaca serta memperbaiki kesalahan inputnya.</p>                                                                                                                                                                                                                                                              |
| 5  | <p><b>Soal:</b> Mengapa validasi sisi klien (<b>client-side validation</b>) menggunakan JavaScript hanya dianggap sebagai lapisan kenyamanan pengguna (<b>user experience</b>) dan bukan sebagai lapisan keamanan data utama?</p> <p><b>Jawaban:</b></p> <p>Validasi sisi klien dieksekusi seluruhnya di dalam lingkungan peramban pengguna (<b>untrusted execution environment</b>). Kode JavaScript berada di bawah kontrol penuh pengguna dan dapat dengan sangat mudah dilewati melalui berbagai teknik: mematikan fitur JavaScript di browser, memanipulasi kode lewat DevTools Console, menghapus atribut validasi via Elements Inspector, atau mengirimkan request payload secara langsung menggunakan perangkat lunak cURL, Postman, atau skrip otomatisasi. Oleh karena itu, validasi client-side hanya berfungsi meningkatkan kenyamanan antarmuka (<b>immediate feedback UX</b>). Keamanan data yang sejati wajib dijamin oleh <b>Validasi Sisi Server (Server-Side Validation)</b> yang tidak dapat dimanipulasi oleh klien, yang akan diimplementasikan secara komprehensif mulai Jobsheet 7.</p> |

# **LAPORAN PRAKTIKUM**

## **JOB SHEET 6 - ASINKRON KOMUNIKASI FETCH API DAN JSON**



**Disusun Oleh:**

Nama : Mohammad Hafidz Rafi' Rabbani  
NIM : 254107020084  
Kelas : 2H  
No. Absen : 14  
Repository : [github.com/hafidzrafi/DnPWeb2026](https://github.com/hafidzrafi/DnPWeb2026)

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**JURUSAN TEKNOLOGI INFORMASI**  
**POLITEKNIK NEGERI MALANG**  
**2026**

# Daftar Isi

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>1. Tujuan .....</b>                                                            | <b>3</b>  |
| <b>2. Praktikum .....</b>                                                         | <b>3</b>  |
| 2.1. Percobaan 1: Pengelolaan Status Pemuatan (Loading State) .....               | 3         |
| 2.1.1. 1.1 Perancangan Elemen Loading Indicator .....                             | 3         |
| 2.1.2. 1.2 Signifikansi Indikator bagi Pengalaman Pengguna (UX) .....             | 4         |
| 2.1.3. Langkah-langkah .....                                                      | 4         |
| 2.1.4. Pertanyaan .....                                                           | 6         |
| 2.2. Percobaan 2: Pengambilan dan Perenderan Dinamis Data Buku .....              | 7         |
| 2.2.1. 2.1 Alur Kerja Fetch API dan JSON Parsing .....                            | 7         |
| 2.2.2. Langkah-langkah .....                                                      | 8         |
| 2.2.3. Pertanyaan .....                                                           | 9         |
| 2.3. Percobaan 3: Pengambilan dan Perenderan Dinamis Data Anggota .....           | 10        |
| 2.3.1. 3.1 Konsistensi Pola Pemrograman Asinkron .....                            | 10        |
| 2.3.2. Langkah-langkah .....                                                      | 11        |
| 2.3.3. Pertanyaan .....                                                           | 12        |
| 2.4. Percobaan 4: Penerapan Event Delegation pada Elemen Dinamis .....            | 13        |
| 2.4.1. 4.1 Kegagalan Pendekatan QuerySelector Konvensional .....                  | 13        |
| 2.4.2. 4.2 Solusi Event Delegation Mengandalkan Event Bubbling .....              | 13        |
| 2.4.3. Langkah-langkah .....                                                      | 14        |
| 2.4.4. Pertanyaan .....                                                           | 15        |
| 2.5. Percobaan 5: Penanganan Kesalahan Komunikasi Jaringan (Error Handling) . . . | 16        |
| 2.5.1. 5.1 Isolasi Kegagalan dengan try...catch .....                             | 16        |
| 2.5.2. 5.2 Penyajian Pesan Kesalahan Terpadu .....                                | 16        |
| 2.5.3. Langkah-langkah .....                                                      | 17        |
| 2.5.4. Pertanyaan .....                                                           | 18        |
| <b>3. Tugas .....</b>                                                             | <b>19</b> |
| 3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 6 .....                        | 19        |

## 1. Tujuan

- Memahami konsep dasar arsitektur komunikasi asinkron berbasis AJAX (Asynchronous JavaScript and XML) dan format serialisasi data JSON (JavaScript Object Notation).
- Menguasai pemanggilan antarmuka pemrograman jaringan modern menggunakan Fetch API berbasis Promise dan sintaksis `async / await`.
- Mengimplementasikan pemisahan lapisan data (data layer) dari struktur dokumen presentasi dengan mengosongkan tag `<tbody>` pada dokumen HTML.
- Membangun mekanisme umpan balik visual status pemuatan antarmuka (loading indicator) selama latensi transfer jaringan berlangsung.
- Menerapkan penanganan kegagalan tangguh (resilient error handling) memanfaatkan blok `try...catch...finally` guna mencegah kegagalan fatal pada antarmuka pengguna.
- Menguasai teknik Event Delegation pada objek tingkat tinggi dokumen (document) untuk menangani elemen antarmuka yang terkonstruksi secara dinamis pasca-DOMContentLoaded.
- Menganalisis batasan keamanan peramban modern terkait kebijakan Same-Origin Policy (SOP) dan Cross-Origin Resource Sharing (CORS) saat berinteraksi dengan berkas lokal.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: <https://github.com/hafidzrafi/DnPWeb2026>.

## 2. Praktikum

### 2.1. Percobaan 1: Pengelolaan Status Pemuatan (Loading State)

Pada percobaan pertama, antarmuka daftar buku dan anggota dipersiapkan untuk menghadapi latensi komunikasi jaringan asinkron. Berbeda dengan halaman web statis di mana seluruh konten langsung tersedia saat file dimuat, operasi jaringan membutuhkan waktu bervariasi tergantung kondisi koneksi.

#### 2.1.1. 1.1 Perancangan Elemen Loading Indicator

Sebuah paragraf penanda status `#loading-indicator` disisipkan di atas tabel dengan gaya awal tersembunyi (`display: none`):

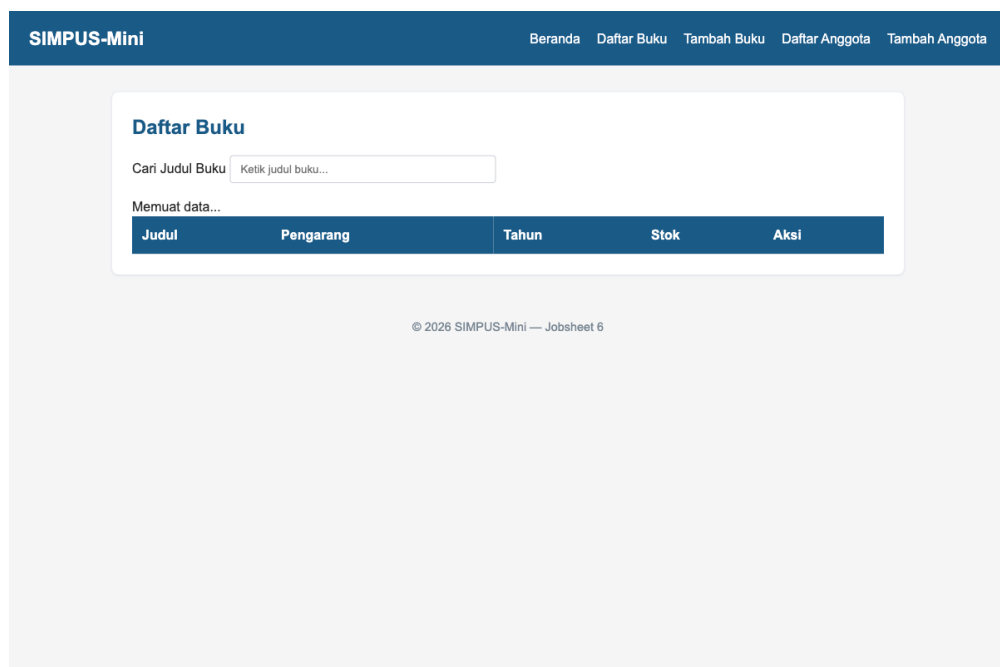
```
<p id="loading-indicator" style="display:none;">Memuat data...</p>
```

Di dalam fungsi pengendali data, indikator ini langsung diaktifkan dengan mengubah properti tampilan menjadi `loading.style.display = "block"` tepat sebelum permintaan jaringan `fetch()` dieksekusi. Untuk menyimulasikan jeda jaringan nyata pada pengujian lokal, digunakan penundaan buatan selama 600 milidetik menggunakan `Promise`:

```
await new Promise((resolve) => setTimeout(resolve, 600));
```

### 2.1.2. 1.2 Signifikansi Indikator bagi Pengalaman Pengguna (UX)

Tanpa adanya umpan balik visual, pengguna dapat mengasumsikan bahwa aplikasi membeku (**frozen**) atau tabel dalam kondisi kosong permanen. Teks “Memuat data...” memberikan kepastian bahwa sistem sedang aktif memproses permintaan di latar belakang.



**Gambar 1.1:** Indikator Status “Memuat data...” Tampil Sesaat Selagi Proses Asinkron Berlangsung.

### 2.1.3. Langkah-langkah

1. Menambahkan elemen `<p id="loading-indicator" style="display:none;">Memuat data...</p>` pada `books/list.html` dan `members/list.html`.
2. Mengosongkan seluruh isi tag `<tbody>` pada kedua tabel HTML.
3. Menambahkan logika aktivasi indikator pada blok awal fungsi `muatDaftarBuku()` dan menonaktifkan pada blok `finally`.

4. Memverifikasi kemunculan teks loading saat halaman pertama kali dibuka sebelum data dirender.

#### 2.1.4. Pertanyaan

1. Apa peran vital dari blok `finally` dalam penanganan indikator pemuatan?
  - Blok `finally` menjamin operasi pembersihan (**cleanup operations**) pasti dieksekusi dalam kondisi apa pun, baik permintaan jaringan berhasil tuntas di blok `try` maupun terhenti akibat kegagalan koneksi di blok `catch`. Hal ini mencegah indikator loading tergantung atau berputar selamanya di layar antarmuka pengguna ketika terjadi error.

## 2.2. Percobaan 2: Pengambilan dan Perenderan Dinamis Data Buku

Percobaan kedua merupakan inti dari arsitektur **Client-Side Rendering (CSR)** pada Jobsheet 6. Seluruh baris data buku yang sebelumnya ditulis secara manual di HTML kini dimuat secara asinkron dari berkas terstruktur `data/buku.json`.

### 2.2.1. 2.1 Alur Kerja Fetch API dan JSON Parsing

Fungsi `muatDaftarBuku()` pada berkas `assets/js/buku.js` mengeksekusi urutan operasi berikut:

```
async function muatDaftarBuku() {
  const tbody = document.querySelector(".table-responsive table tbody");
  const loading = document.getElementById("loading-indicator");
  if (!tbody) return;

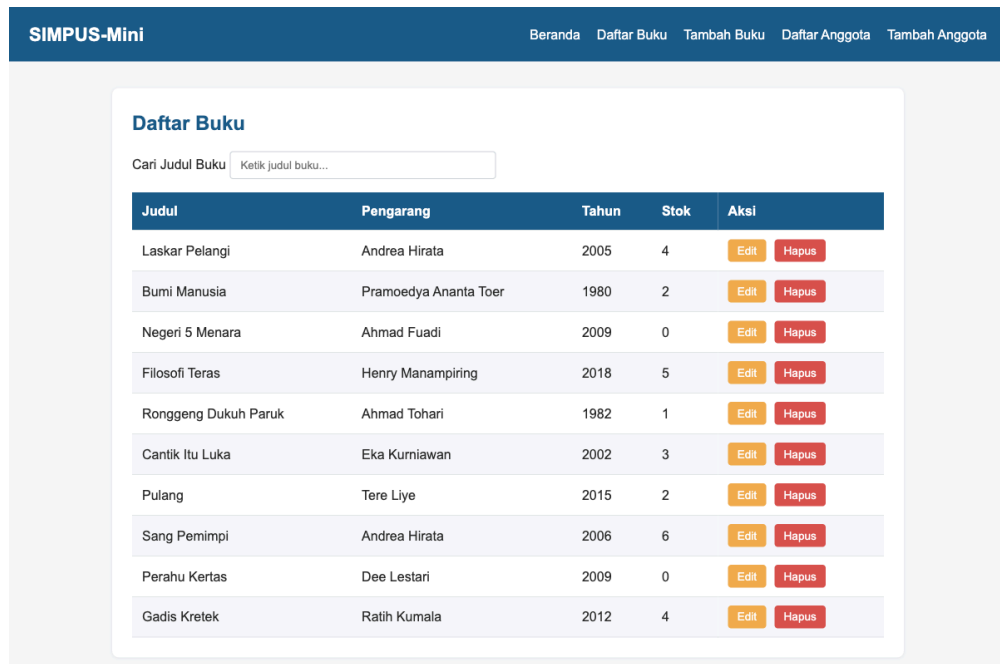
  loading.style.display = "block";
  tbody.innerHTML = "";

  try {
    await new Promise((resolve) => setTimeout(resolve, 600));

    const res = await fetch("../data/buku.json");
    if (!res.ok) {
      throw new Error("Gagal mengambil data (status " + res.status + ")");
    }
    const daftarBuku = await res.json();

    daftarBuku.forEach(function (buku) {
      const tr = document.createElement("tr");
      tr.innerHTML =
        "<td>" + buku.judul + "</td>" +
        "<td>" + buku.pengarang + "</td>" +
        "<td>" + buku.tahun + "</td>" +
        "<td>" + buku.stok + "</td>" +
        "<td>" +
        "<button type='button'>Edit</button> " +
        "<button type='button' class='btn-hapus'>Hapus</button>" +
        "</td>";
      tbody.appendChild(tr);
    });
  } catch (err) {
    tbody.innerHTML =
      "<tr><td colspan='5'>Gagal memuat data: " + err.message + "</td></tr>";
  } finally {
    loading.style.display = "none";
  }
}
```

Method `fetch()` mengembalikan objek `Response`. Pengecekan manual `res.ok` memvalidasi bahwa kode status HTTP berada pada rentang 200–299. Selanjutnya, method `res.json()` mem-parse string format JSON menjadi larik (**array**) objek JavaScript asli. Setiap objek buku diiterasi menggunakan `forEach()` untuk membangun elemen baris `<tr>` baru dan dimasukkan ke dalam `<tbody>` memanfaatkan `tbody.appendChild(tr)`.



| Judul                | Pengarang             | Tahun | Stok | Aksi       |
|----------------------|-----------------------|-------|------|------------|
| Laskar Pelangi       | Andrea Hirata         | 2005  | 4    | Edit Hapus |
| Bumi Manusia         | Pramoedya Ananta Toer | 1980  | 2    | Edit Hapus |
| Negeri 5 Menara      | Ahmad Fuadi           | 2009  | 0    | Edit Hapus |
| Filosofi Teras       | Henry Manampiring     | 2018  | 5    | Edit Hapus |
| Ronggeng Dukuh Paruk | Ahmad Tohari          | 1982  | 1    | Edit Hapus |
| Cantik Itu Luka      | Eka Kurniawan         | 2002  | 3    | Edit Hapus |
| Pulang               | Tere Liye             | 2015  | 2    | Edit Hapus |
| Sang Pemimpi         | Andrea Hirata         | 2006  | 6    | Edit Hapus |
| Perahu Kertas        | Dee Lestari           | 2009  | 0    | Edit Hapus |
| Gadis Kretek         | Ratih Kumala          | 2012  | 4    | Edit Hapus |

**Gambar 1.2:** Hasil Konsumsi Asinkron `data/buku.json` yang Dirender Menjadi 10 Baris Tabel Dinamis.

### 2.2.2. Langkah-langkah

1. Menyusun berkas `data/buku.json` berisi 10 rekaman objek buku lengkap dengan atribut judul, pengarang, tahun, dan stok.
2. Mengimplementasikan fungsi `muatDaftarBuku()` pada berkas `assets/js/buku.js`.
3. Memasang pemanggilan fungsi otomatis saat event `'DOMContentLoaded'` terpicu.
4. Memverifikasi melalui browser bahwa 10 data buku terrender sempurna ke dalam tabel tanpa pemuatan ulang halaman.

### 2.2.3. Pertanyaan

1. Mengapa properti `res.ok` harus diperiksa secara manual sebelum menjalankan `res.json()`?
  - Karena Fetch API dirancang hanya me-reject Promise jika terjadi kegagalan jaringan fatal tingkat transport (seperti jaringan offline atau DNS gagal diresolusi). Apabila server mengembalikan status error HTTP seperti 404 (Not Found) atau 500 (Internal Server Error), Fetch API tetap menganggapnya sebagai resolusi berhasil. Oleh karena itu, jika `!res.ok` tidak ditangkap secara eksplisit, eksekusi `res.json()` terhadap payload halaman HTML error 404 akan memicu **SyntaxError: Unexpected token**.

## 2.3. Percobaan 3: Pengambilan dan Perenderan Dinamis Data Anggota

Percobaan ketiga menerapkan arsitektur serupa pada modul anggota perpustakaan menggunakan berkas data `data/anggota.json` dan skrip pengendali `assets/js/anggota.js`.

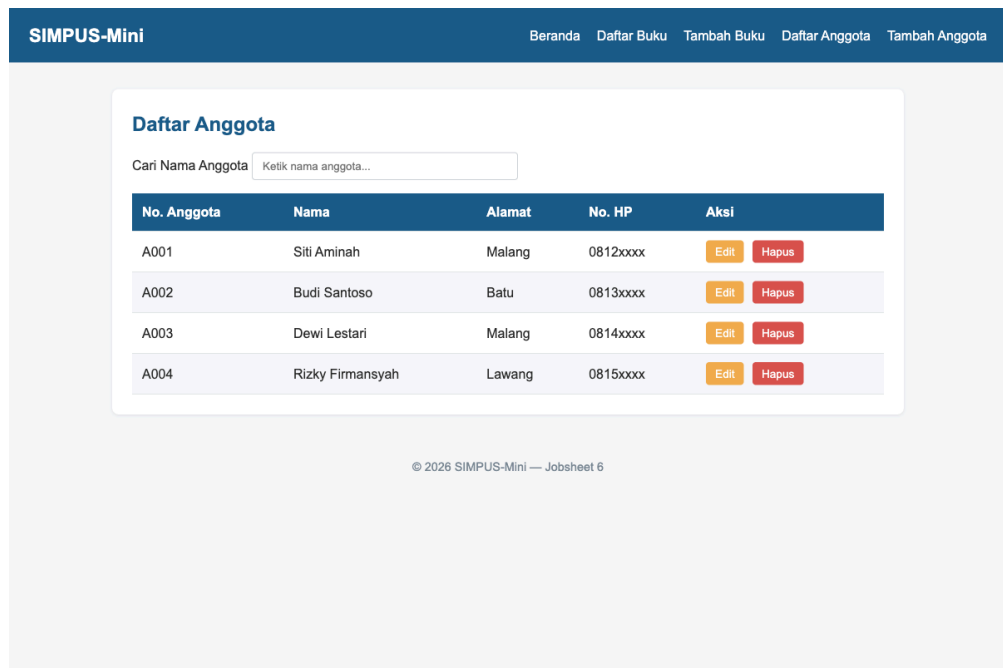
### 2.3.1. 3.1 Konsistensi Pola Pemrograman Asinkron

Fungsi `muatDaftarAnggota()` membaca data 4 orang anggota perpustakaan yang mencakup atribut nomor anggota (`no_anggota`), nama lengkap, alamat tempat tinggal, dan nomor telepon seluler:

```
async function muatDaftarAnggota() {
  // ...inisialisasi loading & pembersihan tbody...
  const res = await fetch("../data/anggota.json");
  if (!res.ok) throw new Error("Gagal mengambil data (status " + res.status + ")");
  const daftarAnggota = await res.json();

  daftarAnggota.forEach(function (anggota) {
    const tr = document.createElement("tr");
    tr.innerHTML =
      "<td>" + anggota.no_anggota + "</td>" +
      "<td>" + anggota.nama + "</td>" +
      "<td>" + anggota.alamat + "</td>" +
      "<td>" + anggota.no_hp + "</td>" +
      "<td>" +
      "<button type='button'>Edit</button> " +
      "<button type='button' class='btn-hapus'>Hapus</button>" +
      "</td>";
    tbody.appendChild(tr);
  });
}
```

Penerapan pola ini membuktikan fleksibilitas konsep perenderan dinamis sisi klien, di mana struktur logika kode yang seragam dapat menyajikan berbagai skema data berbeda hanya dengan menyesuaikan struktur JSON sumbernya.



**Gambar 1.3:** Tabel Anggota Perpustakaan Berhasil Memuat 4 Rekaman Data Secara Dinamis dari `data/anggota.json`.

### 2.3.2. Langkah-langkah

1. Menyusun berkas ``data/anggota.json`` dengan data anggota perpustakaan.
2. Mengembangkan skrip ``assets/js/anggota.js`` dan menautkannya pada ``members/list.html``.
3. Menguji perenderan tabel anggota melalui peramban web dan memeriksa struktur elemen melalui tab Elements DevTools.

### 2.3.3. Pertanyaan

1. Bagaimana struktur arsitektur ini mempermudah transisi menuju integrasi RESTful API di masa mendatang?
  - Pada struktur ini, kode antarmuka JavaScript telah sepenuhnya terisolasi dari format penyimpanan data. Di masa depan, pengembang hanya perlu mengubah URL target pemanggilan `fetch("../data/anggota.json")` menjadi endpoint API backend sungguhan (misalnya `fetch("/api/anggota")`) tanpa perlu merombak logika perenderan DOM dan tata letak tabel antarmuka pengguna.

## 2.4. Percobaan 4: Penerapan Event Delegation pada Elemen Dinamis

Percobaan keempat mengatasi masalah fundamental dalam pemrograman berbasis kejadian (**event-driven programming**) saat berhadapan dengan elemen yang dibuat secara dinamis setelah dokumen selesai dimuat.

### 2.4.1. 4.1 Kegagalan Pendekatan QuerySelector Konvensional

Pada Jobsheet 5, tombol hapus dipasang penangan event menggunakan:

```
document.querySelectorAll(".btn-hapus").forEach(...)
```

Pendekatan di atas mengalami kegagalan total (**silent failure**) pada Jobsheet 6. Karena baris tabel dibuat secara asinkron beberapa ratus milidetik setelah inisialisasi awal, pemanggilan `querySelectorAll` saat `DOMContentLoaded` hanya menemukan elemen kosong (`NodeList(0)`). Akibatnya, tombol-tombol Hapus yang baru muncul tidak memiliki listener terpasang.

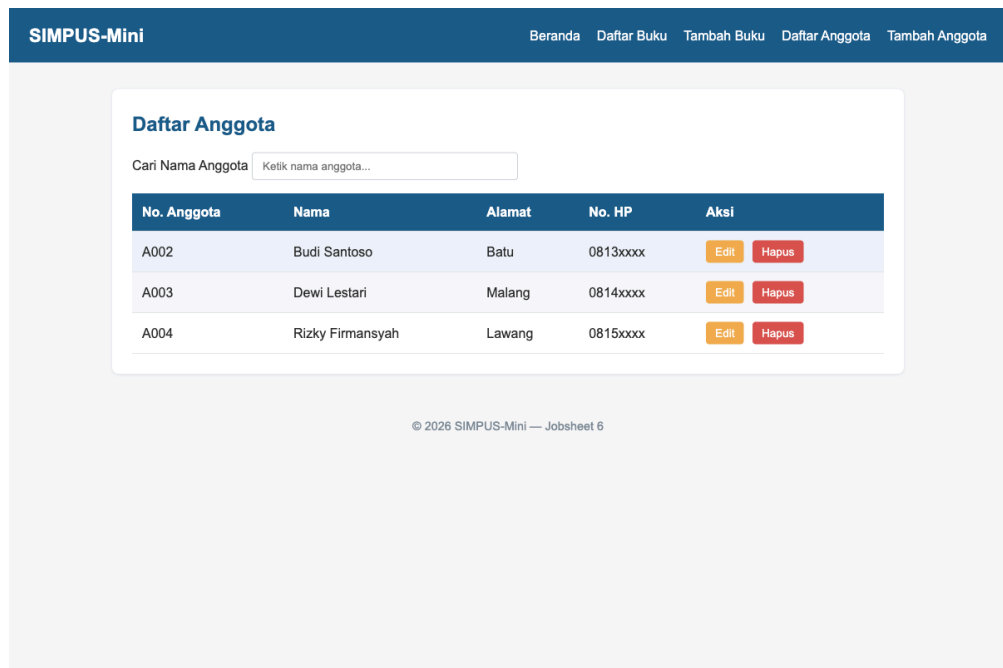
### 2.4.2. 4.2 Solusi Event Delegation Mengandalkan Event Bubbling

Solusi rekayasa yang tepat adalah **Event Delegation**, yaitu mendaftarkan satu-satunya event listener pada objek tingkat teratas dokumen (`document`):

```
function initHapusConfirm() {
  document.addEventListener("click", function (e) {
    const btn = e.target.closest(".btn-hapus");
    if (!btn) return;

    const row = btn.closest("tr");
    const nama = row ? row.querySelector("td").textContent : "data ini";
    const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
    if (yakin && row) {
      row.remove();
    }
  });
}
```

Ketika pengguna mengklik elemen apa pun di layar, event klik akan merambat naik menuju root dokumen (**Event Bubbling**). Method `e.target.closest(".btn-hapus")` menyaring kejadian tersebut: jika klik tidak berasal dari tombol hapus atau turunannya, fungsi langsung mengembalikan nilai (**early return**). Jika cocok, dialog konfirmasi diaktifkan dan baris target dicabut dari DOM.



**Gambar 1.4:** Baris Data Anggota Pertama (A001) Berhasil Dihapus Menggunakan Pendekatan Event Delegation.

#### 2.4.3. Langkah-langkah

1. Memodifikasi implementasi fungsi ``initHapusConfirm()`` di dalam ``assets/js/app.js`` menggunakan ``document.addEventListener("click", ...)``.
2. Menggunakan ``e.target.closest(".btn-hapus")`` untuk mendeteksi target interaksi.
3. Menguji klik tombol Hapus pada baris buku dan anggota yang dibuat secara dinamis.
4. Memverifikasi bahwa dialog konfirmasi tetap muncul dan baris yang dipilih berhasil terhapus dari DOM.

#### 2.4.4. Pertanyaan

1. Sebutkan dua keuntungan utama penggunaan Event Delegation dari sudut pandang efisiensi memori!
  - 1. **Optimalisasi Penggunaan Memori:** Mencegah pembuatan ratusan objek listener fungsi di memori peramban. Berapa pun jumlah baris yang ada di dalam tabel (bahkan ribuan), peramban hanya menyimpan satu buah listener di level dokumen.
  - 2. **Ketahanan Elemen Dinamis:** Setiap elemen baru yang ditambahkan ke dalam DOM di kemudian hari secara otomatis langsung dapat berinteraksi tanpa memerlukan registrasi listener ulang.

## 2.5. Percobaan 5: Penanganan Kesalahan Komunikasi Jaringan (Error Handling)

Percobaan kelima menguji ketahanan aplikasi saat menghadapi kegagalan jaringan atau berkas sumber data yang tidak ditemukan (**graceful degradation**).

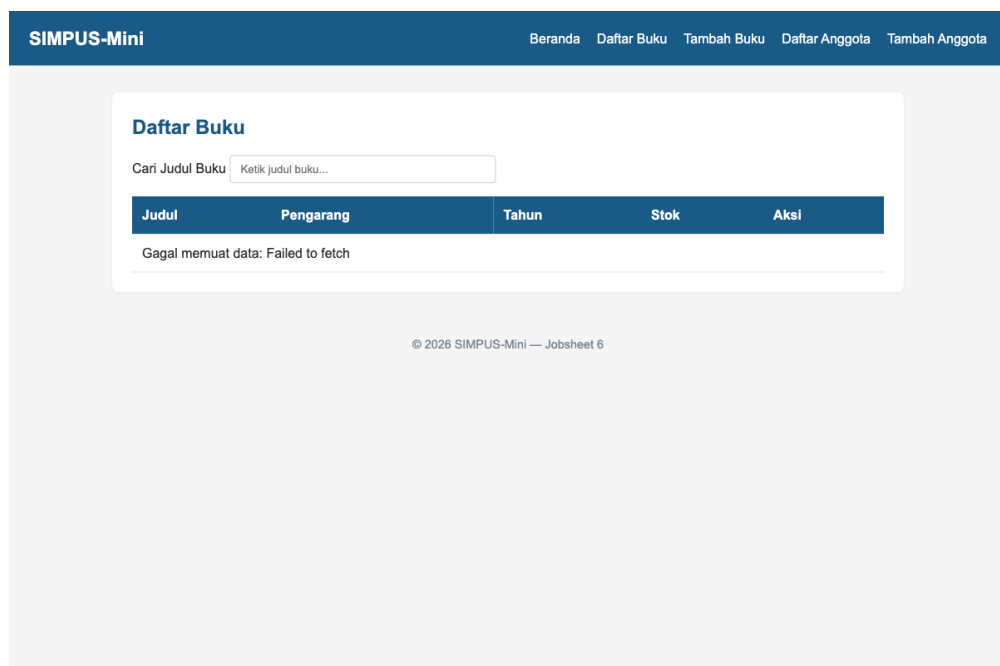
### 2.5.1. 5.1 Isolasi Kegagalan dengan try...catch

Blok `try...catch` disematkan untuk memayungi seluruh operasi kritis. Apabila permintaan `fetch()` mengalami kegagalan (misalnya koneksi server offline atau nama berkas salah), alur program tidak terhenti, melainkan langsung ditangkap oleh blok `catch (err)`:

```
try {  
    // ...operasi fetch...  
} catch (err) {  
    tbody.innerHTML =  
        "<tr><td colspan='5'>Gagal memuat data: " + err.message + "</td></tr>";  
} finally {  
    loading.style.display = "none";  
}
```

### 2.5.2. 5.2 Penyajian Pesan Kesalahan Terpadu

Daripada membiarkan halaman rusak atau menampilkan konsol error merah tanpa informasi visual, aplikasi secara cerdas mengganti konten `<tbody>` dengan sebuah baris tunggal berspan 5 kolom yang menginformasikan rincian galat kepada pengguna.



**Gambar 1.5:** Pesan Kesalahan Ditampilkan Elegan di Dalam Tabel Saat Terjadi Kegagalan Jaringan.

### **2.5.3. Langkah-langkah**

1. Mengimplementasikan blok penanganan ``try...catch...finally`` pada fungsi pembacaan data.
2. Menguji skenario kegagalan jaringan (misal mengubah sementara path file target fetch atau menghentikan server).
3. Mengamati tabel menampilkan pesan error yang ramah pengguna dan indikator loading tertutup rapi.

#### 2.5.4. Pertanyaan

1. Mengapa aplikasi web profesional tidak boleh membiarkan error jaringan hanya tercatat di Console browser?
  - Pengguna akhir (**end-users**) aplikasi web tidak membuka tab Developer Tools Console. Jika error jaringan hanya dicatat di konsol, pengguna akan melihat antarmuka kosong yang membingungkan dan mengira aplikasi mengalami **hang**. Menampilkan pesan kesalahan informatif di antarmuka memberi tahu pengguna situasi yang sebenarnya terjadi dan langkah yang perlu diambil (misalnya memeriksa koneksi internet atau memuat ulang halaman).

### 3. Tugas

#### 3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 6

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pokok pertanyaan konseptual pada Jobsheet 6:

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <p><b>Soal:</b> Apa keuntungan utama arsitektur komunikasi data asinkron (<b>asynchronous communication</b>) menggunakan Fetch API dibandingkan pemuatan ulang halaman web secara menyeluruh (<b>full page reload</b>)?</p> <p><b>Jawaban:</b></p> <ol style="list-style-type: none"><li>1. <b>Efisiensi Beban Jaringan (Payload Bandwidth):</b> Pada pemuatan ulang konvensional, server harus mentransmisikan ulang seluruh kerangka dokumen HTML, stylesheet CSS, skrip JavaScript, serta aset media. Dengan Fetch API asinkron, transfer data hanya terbatas pada muatan data mentah terstruktur (format JSON) berukuran sangat ringkas, menghemat bandwidth secara masif.</li><li>2. <b>Kontinuitas Pengalaman Pengguna (Smooth UX):</b> Layar peramban tidak mengalami fenomena kedipan putih (<b>white flash</b>) maupun kehilangan status scroll dan posisi fokus saat data diperbarui.</li><li>3. <b>Pemisahan Tugas Arsitektur:</b> Mengukuhkan arsitektur modern di mana server web murni berperan sebagai penyedia data (<b>API Provider</b>), sementara logika presentasi grafis didelegasikan ke mesin peramban pengguna (<b>Client-Side Rendering</b>).</li></ol> |
| 2  | <p><b>Soal:</b> Mengapa eksekusi fungsi <code>fetch()</code> selalu mengembalikan objek Promise dan harus dipanggil menggunakan kata kunci <code>await</code> di dalam fungsi <code>async</code>? Mengapa properti <code>res.ok</code> tetap wajib diverifikasi secara manual sebelum mem-parsing data JSON?</p> <p><b>Jawaban:</b></p> <p>Fungsi <code>fetch()</code> mengembalikan objek <code>Promise</code> karena transmisi data jaringan bersifat <b>I/O-bound</b> dan membutuhkan waktu tunggu (<b>latency</b>)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>yang tidak deterministik. Mengingat mesin JavaScript beroperasi pada <b>single-threaded event loop</b>, proses penantian tersebut harus berjalan secara non-pemblokir (<b>non-blocking</b>). Sintaksis <code>async/await</code> menyediakan abstraksi elegan agar penulisan logika asinkron dapat dibaca secara sekuensial lurus tanpa terjebak dalam perangkat <b>callback hell</b>.</p> <p>Properti <code>res.ok</code> (bernilai <code>true</code> jika status kode HTTP berada di rentang 200–299) wajib diverifikasi manual karena arsitektur bawaan Fetch API <b>hanya akan me-reject Promise jika terjadi kegagalan jaringan fisik</b> (misalnya koneksi putus atau server mati total). Ketika server mengembalikan status galat HTTP 404 (Not Found) atau 500 (Internal Server Error), Fetch API tetap memperlakukannya sebagai operasi sukses (<b>fulfilled Promise</b>). Tanpa pemeriksaan <code>!res.ok</code>, pemanggilan berikutnya <code>res.json()</code> terhadap badan halaman HTML galat 404 akan memicu kesalahan fatal <b>SyntaxError: Unexpected token</b> saat proses parsing dilakukan.</p> |
| 3  | <p><b>Soal:</b> Bagaimana struktur penanganan kegagalan <code>try...catch...finally</code> menjaga aplikasi tetap <b>resilient</b> (tangguh) saat terjadi kegagalan jaringan atau berkas tidak ditemukan? Apa peran spesifik dari blok <code>finally</code> pada kasus <b>loading indicator</b>?</p> <p><b>Jawaban:</b></p> <p>Struktur <code>try...catch...finally</code> mengisolasi blok kode rawan kegagalan di dalam <code>try</code>. Ketika kegagalan jaringan terdeteksi atau pengecualian dilempar melalui pernyataan <code>throw new Error(...)</code>, aliran komputasi langsung dialihkan ke blok <code>catch(err)</code> untuk menampilkan komponen pesan kegagalan ramah pengguna di dalam tabel, menjaga agar seluruh sistem aplikasi tetap beroperasi secara anggun tanpa mengalami <b>runtime crash</b>.</p> <p>Peran spesifik dari blok <code>finally</code> adalah menjalankan prosedur pembersihan mutlak (<b>mandatory cleanup</b>). Dalam skenario antarmuka ini, menyembunyikan indikator <code>loading.style.display = "none"</code> ditempatkan di dalam blok <code>finally</code></p>        |

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>untuk memastikan bahwa indikator pemuatan pasti disembunyikan kembali tanpa memandang apakah operasi <code>try</code> berakhir dengan keberhasilan data atau kegagalan yang ditangkap oleh <code>catch</code>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 4  | <p><b>Soal:</b> Mengapa pada Jobsheet 6 penanganan event klik tombol Hapus (<code>.btn-hapus</code>) wajib dialihkan menggunakan teknik <b>Event Delegation</b> pada objek leluhur (<code>document</code>) dan tidak bisa lagi menggunakan <code>querySelectorAll(".btn-hapus").forEach(...)</code> seperti pada Jobsheet 5?</p> <p><b>Jawaban:</b></p> <p>Pada Jobsheet 5, seluruh baris tabel ditulis secara statis di dokumen HTML sehingga seluruh tombol <code>.btn-hapus</code> telah eksis di memori DOM saat event <code>DOMContentLoaded</code> selesai terpicu. Sebaliknya, pada Jobsheet 6, penampung <code>&lt;tbody&gt;</code> berada dalam keadaan kosong saat pembacaan skrip dimulai. Elemen-elemen baris tabel baru diinjeksikan ke dalam pohon DOM beberapa saat kemudian setelah penyelesaian request jaringan asinkron. Penggunaan metode <code>querySelectorAll</code> pada tahap awal hanya akan menghasilkan koleksi elemen kosong (<code>NodeList(0)</code>), menyebabkan seluruh tombol Hapus dinamis tidak memiliki penanganan event klik.</p> <p>Teknik <b>Event Delegation</b> menempatkan satu penanganan event tunggal pada node leluhur abadi (<code>document</code>). Mengandalkan fase perambatan kejadian ke atas (<b>Event Bubbling</b>), setiap klik pada area mana pun di layar akan tereskalasi ke tingkat dokumen. Penyeleksian berbasis <code>e.target.closest(".btn-hapus")</code> kemudian menyaring interaksi tersebut secara efisien: mengabaikan klik pada area lain dan mengeksekusi konfirmasi penghapusan saat tombol Hapus dinamis ditekan.</p> |
| 5  | <p><b>Soal:</b> Mengapa pemanggilan fungsi <code>fetch()</code> ke berkas lokal (<code>buku.json</code> / <code>anggota.json</code>) otomatis diblokir oleh peramban ketika dibuka menggunakan protokol berkas lokal (<code>file://</code>), dan mengapa peramban membutuhkan protokol <code>http://</code> dari server lokal?</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

| No | Pertanyaan dan Jawaban Konseptual                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p><b>Jawaban:</b></p> <p>Peramban web modern menerapkan mekanisme pertahanan keamanan fundamental bernama <b>Same-Origin Policy (SOP)</b> dan pembatasan <b>Cross-Origin Resource Sharing (CORS)</b>. Ketika halaman web dibuka secara langsung melalui sistem berkas lokal (<code>file://</code>), peramban memperlakukan origin berkas tersebut sebagai <b>opaque / null origin</b>. Untuk mencegah skrip berbahaya menyusup dan membaca berkas lokal sensitif di media penyimpanan komputer pengguna tanpa izin, peramban secara otomatis memblokir seluruh komunikasi I/O asinkron antar berkas pada protokol <code>file://</code>.</p> <p>Dengan menjalankan server web lokal (seperti <code>php -S localhost:8000</code>), seluruh berkas HTML dan berkas JSON disajikan di bawah identitas origin HTTP resmi yang seragam (<code>http://localhost:8000</code>). Keberadaan origin yang sah ini memenuhi kriteria otorisasi keamanan peramban sehingga eksekusi pertukaran data asinkron via Fetch API diizinkan berjalan normal.</p> |